

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«процедуры, функции, триггеры в PostgreSQL» по дисциплине
«Проектирование и реализация баз данных»
Вариант 2. БД «Сессия»**

Обучающийся Иванова Аайа Гаврильевна
Факультет прикладной информатики
Группа К3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД PostgreSQL, SQL Shell (psql).

Практическое задание (min - 6 баллов, max - 10 баллов, доп. баллы - 3):

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры. (3 балла)
2. Создать триггеры для индивидуальной БД согласно варианту:

Вариант 2.2. 5 оригинальных триггеров + модификация триггерной функции Части 2 практического занятия доп. баллы).

Дополнительные баллы - 3:

Модифицировать триггер (триггерную функцию) на проверку корректности входа и выхода сотрудника (см. Практическое задание 1 Лабораторного практикума (Приложение)) с максимальным учетом «узких» мест некорректных данных по входу и выходу).

Указание. Работа выполняется в консоли SQL Shell (psql).

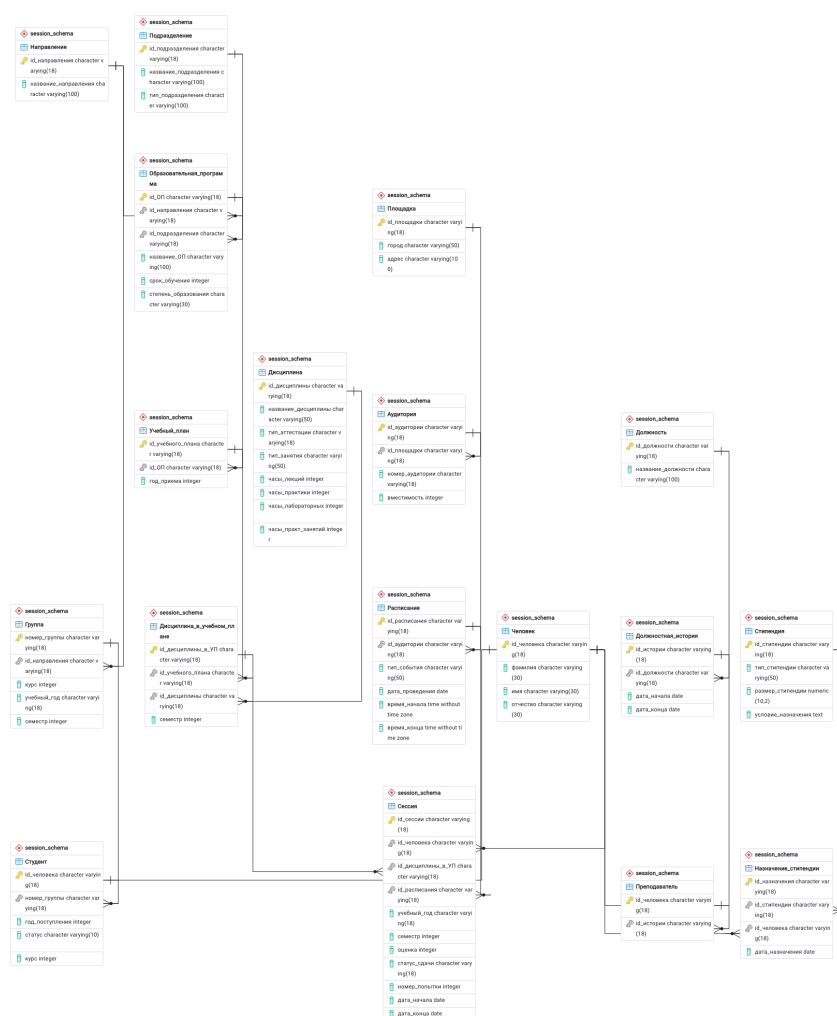


Рисунок 1 – Схема логической модели базы данных, сгенерированная в Generate ERD

Процедуры:

1. Зачисление студента на пересдачу экзамена.
- Создаем процедуру:

```
CREATE OR REPLACE PROCEDURE session_schema.register_retake(
    IN p_id_cheloveka VARCHAR(18),
    IN p_id_discipliny_v_up VARCHAR(18),
    IN p_id_raspisaniya VARCHAR(18),
    IN p_uchebnyy_god VARCHAR(18),
    IN p_semestr INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_max_popytka INTEGER;
    v_student_status VARCHAR(10);
    v_disciplina_exists INTEGER;
    v_new_popytka INTEGER;
    v_familia VARCHAR(30);
    v_disciplina_name VARCHAR(50);
    v_new_id VARCHAR(18);
BEGIN
    -- Проверяем статус студента
    SELECT статус INTO v_student_status
    FROM session_schema.Студент
    WHERE id_человека = p_id_cheloveka;

    IF v_student_status IS NULL THEN
        RAISE EXCEPTION 'Студент с ID % не найден', p_id_cheloveka;
    END IF;

    IF v_student_status = 'отчислен' THEN
        RAISE EXCEPTION 'Студент % уже отчислен и не может быть зачислен на
пересдачу', p_id_cheloveka;
    END IF;

    -- Проверяем наличие дисциплины в учебном плане
    SELECT COUNT(*) INTO v_disciplina_exists
    FROM session_schema.Дисциплина_в_учебном_плане
    WHERE id_дисциплины_в_УП = p_id_discipliny_v_up;

    IF v_disciplina_exists = 0 THEN
        RAISE EXCEPTION 'Дисциплина не найдена в учебном плане';
    END IF;

    -- Проверяем существование расписания
    IF p_id_raspisaniya IS NULL THEN
        RAISE EXCEPTION 'Не указано расписание для пересдачи';
    END IF;

    -- Находим максимальный номер попытки
    SELECT COALESCE(MAX(номер_попытки), 0) INTO v_max_popytka
    FROM session_schema.Сессия
```

```

WHERE id_человека = p_id_cheloveka
AND id_дисциплины_в_УП = p_id_discipliny_v_up;

IF v_max_popytka >= 3 THEN
    RAISE EXCEPTION 'Превышено максимальное количество попыток (3) для сдачи
дисциплины';
END IF;

v_new_popytka := v_max_popytka + 1;

-- Генерируем короткий ID
v_new_id := 'RET_' || p_id_cheloveka || '_' || v_new_popytka;

-- Получаем фамилию и название дисциплины
SELECT фамилия INTO v_familia
FROM session_schema.Человек
WHERE id_человека = p_id_cheloveka;

SELECT d.название_дисциплины INTO v_disciplina_name
FROM session_schema.Дисциплина_в_учебном_плане dup
JOIN session_schema.Дисциплина d ON dup.id_дисциплины = d.id_дисциплины
WHERE dup.id_дисциплины_в_УП = p_id_discipliny_v_up;

-- Создаем запись о сессии
INSERT INTO session_schema.Сессия (
    id_сессии,
    id_человека,
    id_дисциплины_в_УП,
    id_расписания,
    учебный_год,
    семестр,
    оценка,
    статус_сдачи,
    номер_попытки,
    дата_начала,
    дата_конца
) VALUES (
    v_new_id,
    p_id_cheloveka,
    p_id_discipliny_v_up,
    p_id_raspisaniya,
    p_uchebnyy_god,
    p_semestr,
    NULL,
    'не сдано',
    v_new_popytka,
    CURRENT_DATE,
    CURRENT_DATE
);

RAISE NOTICE 'Студент % зачислен на пересдачу "%" (попытка % из 3, ID: %)',
    v_familia, v_disciplina_name, v_new_popytka, v_new_id;

END;
```

\$\$;

Проверяем текущий список студентов, которые зачислены на пересдачу:

```
DB_session=# SELECT 'Д0: Текущие записи о сессиях' AS этап;
SELECT
    s.id_сессии,
    c.фамилия,
    d.название_дисциплины,
    s.оценка,
    s.номер_попытки
FROM session_schema.Сессия s
JOIN session_schema.Человек c ON s.id_человека = c.id_человека
JOIN session_schema.Дисциплина_в_учебном_плане dup ON s.id_дисциплины_в_УП = dup.id_дисциплины_в_УП
JOIN session_schema.Дисциплина d ON dup.id_дисциплины = d.id_дисциплины
WHERE s.id_человека = 'STU_01'
[ORDER BY s.id_сессии;
    этап
```

Д0: Текущие записи о сессиях
(1 строка)

id_сессии	фамилия	название_дисциплины	оценка	номер_попытки
SES_01	Бэггинс	Базы данных	5	1
SES_03	Бэггинс	Программирование	5	1

(2 строки)

Проверяем доступные расписания:

```
DB_session=# SELECT id_расписания, тип_события, дата_проведения
FROM session_schema.Расписание;
```

id_расписания	тип_события	дата_проведения
SCH_01	Экзамен	2026-01-15
SCH_02	Дифзачёт	2026-01-16
SCH_03	Экзамен	2026-01-18
SCH_04	Зачет	2026-01-19
SCH_05	Экзамен	2026-01-20
SCH_06	Дифзачет	2026-01-21
SCH_07	Экзамен	2026-01-22

(7 строк)

Вызываем процедуру:

```
DB_session=# CALL session_schema.register_retake(
    'STU_01'::VARCHAR,
    'DUP_01'::VARCHAR,
    'SCH_01'::VARCHAR,
    '2025/2026'::VARCHAR,
    5
);
NOTICE: Студент Бэггинс зачислен на пересдачу "Базы данных" (попытка 2 из 3, ID: RET_STU_01_2)
CALL
```

Проверяем новый список:

```

DB_session=# SELECT 'ПОСЛЕ: Все записи о сессиях для STU_01' AS этап;
SELECT
    s.id_сессии,
    c.фамилия,
    d.название_дисциплины,
    s.оценка,
    s.статус_сдачи,
    s.номер_попытки
FROM session_schema.Сессия s
JOIN session_schema.Человек c ON s.id_человека = c.id_человека
JOIN session_schema.Дисциплина_в_учебном_плане dup ON s.id_дисциплины_в_УП = dup.id_дисциплины_в_УП
JOIN session_schema.Дисциплина d ON dup.id_дисциплины = d.id_дисциплины
WHERE s.id_человека = 'STU_01'
ORDER BY s.id_сессии;
          этап

```

ПОСЛЕ: Все записи о сессиях для STU_01
(1 строка)

id_сессии	фамилия	название_дисциплины	оценка	статус_сдачи	номер_попытки
RET_STU_01_2	Бэггинс	Базы данных		не сдано	2
SES_01	Бэггинс	Базы данных	5	сдано	1
SES_03	Бэггинс	Программирование	5	сдано	1

(3 строки)

При тестировании процедуры обнаружено «узкое место»: процедура не проверяет статус предыдущей сдачи дисциплины. Студент, уже сдавший экзамен на положительную оценку, может быть зачислен на пересдачу. Для исправления необходимо добавить проверку наличия успешной сдачи в процедуре.

2. Начисление стипендий по результатам сессии.

Создаем процедуру:

```

CREATE OR REPLACE PROCEDURE session_schema.assign_scholarships(
    IN p_nomer_gruppy VARCHAR(18),
    IN p_semestr INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_student_record RECORD;
    v_avg_grade DECIMAL(4,2);
    v_has_threes BOOLEAN;
    v_scholarship_type VARCHAR(50);
    v_id_scholarship VARCHAR(18);
    v_students_count INTEGER := 0;
    v_academic_count INTEGER := 0;
    v_enhanced_count INTEGER := 0;
    v_social_count INTEGER := 0;
BEGIN
    -- Проверяем существование группы
    IF NOT EXISTS (
        SELECT 1 FROM session_schema.Группа WHERE номер_группы = p_nomer_gruppy
    ) THEN
        RAISE EXCEPTION 'Группа % не найдена', p_nomer_gruppy;
    END IF;

    RAISE NOTICE '=== Начисление стипендий для группы % за семестр % ===',
    p_nomer_gruppy, p_semestr;

    -- Проходим по всем студентам группы

```

```

FOR v_student_record IN
  SELECT
    st.id_человека,
    c.фамилия || ' ' || c.имя AS fio,
    st.статус
  FROM session_schema.Студент st
  JOIN session_schema.Человек c ON st.id_человека = c.id_человека
  WHERE st.номер_группы = p_nomer_gruppy
  AND st.статус = 'обучается'
LOOP
  -- Вычисляем средний балл студента за семестр
  SELECT
    COALESCE(AVG(s.оценка), 0),
    BOOL_OR(s.оценка = 3)
  INTO
    v_avg_grade,
    v_has_threes
  FROM session_schema.Сессия s
  JOIN session_schema.Дисциплина_в_учебном_плане dup ON s.id_дисциплины_в_УП
= dup.id_дисциплины_в_УП
  WHERE s.id_человека = v_student_record.id_человека
  AND s.семестр = p_semestr
  AND s.оценка IS NOT NULL;

  -- Определяем тип стипендии
  IF v_avg_grade >= 4.5 AND NOT v_has_threes THEN
    -- Повышенная стипендия (отличники без троек)
    v_id_scholarship := 'SCH_02';
    v_scholarship_type := 'Повышенная';
    v_enhanced_count := v_enhanced_count + 1;
  ELSIF v_avg_grade >= 3.0 AND NOT v_has_threes THEN
    -- Академическая стипендия (хорошисты и отличники)
    v_id_scholarship := 'SCH_01';
    v_scholarship_type := 'Академическая';
    v_academic_count := v_academic_count + 1;
  ELSE
    -- Социальная стипендия (есть тройки или средний балл < 3)
    v_id_scholarship := 'SCH_03';
    v_scholarship_type := 'Социальная';
    v_social_count := v_social_count + 1;
  END IF;

  -- Удаляем старое назначение стипендии (если есть)
  DELETE FROM session_schema.Назначение_стипендии
  WHERE id_человека = v_student_record.id_человека;

  -- Добавляем новое назначение стипендии
  INSERT INTO session_schema.Назначение_стипендии (
    id_назначения,
    id_стипендии,
    id_человека,
    дата_назначения
  ) VALUES (

```

```

        'APP_' || v_student_record.id_человека || '_' || p_semestr,
        v_id_scholarship,
        v_student_record.id_человека,
        CURRENT_DATE
    );

    v_students_count := v_students_count + 1;

    RAISE NOTICE 'Студент %: средний балл = %, троек = %, стипендия = %',
        v_student_record.fio, v_avg_grade,
        CASE WHEN v_has_threes THEN 'есть' ELSE 'нет' END,
        v_scholarship_type;
END LOOP;

RAISE NOTICE '=== Итого: назначено % стипендий ===', v_students_count;
RAISE NOTICE ' - Академических: %', v_academic_count;
RAISE NOTICE ' - Повышенных: %', v_enhanced_count;
RAISE NOTICE ' - Социальных: %', v_social_count;
END;
$$;

```

Проверяем текущие назначения стипендий:

```

DB_session=# SELECT
    c.фамилия || ' ' || c.имя AS студент,
    st.номер_группы,
    sch.тип_стипендии,
    sch.размер_стипендии,
    app.дата_назначения
FROM session_schema.Назначение_стипендии app
JOIN session_schema.Человек c ON app.id_человека = c.id_человека
JOIN session_schema.Студент st ON c.id_человека = st.id_человека
JOIN session_schema.Стипендия sch ON app.id_стипендии = sch.id_стипендии
WHERE st.номер_группы = 'K3241'
ORDER BY c.фамилия;

```

студент	номер_группы	тип_стипендии	размер_стипендии	дата_назначения
Бэггинс Фродо	K3241	Повышенная	7000.00	2025-09-01
Зеленый Леголас	K3241	Повышенная	7000.00	2025-10-15

(2 строки)

Вызываем процедуру:

```

[DB_session=# CALL session_schema.assign_scholarships('K3241', 5);
NOTICE:  === Начисление стипендий для группы K3241 за семестр 5 ===
NOTICE:  Студент Бэггинс Фродо: средний балл = 5.00, троек = нет, стипендия = Повышенная
NOTICE:  Студент Зеленый Леголас: средний балл = 4.00, троек = нет, стипендия = Академическая
NOTICE:  Студент Торин Дубошит: средний балл = 0.00, троек = нет, стипендия = Социальная
NOTICE:  Студент Арагорн Элессар: средний балл = 5.00, троек = нет, стипендия = Повышенная
NOTICE:  Студент Теоден Эаджисович: средний балл = 4.00, троек = нет, стипендия = Академическая
NOTICE:  Студент Эомер Эаджисович: средний балл = 5.00, троек = нет, стипендия = Повышенная
NOTICE:  Студент Эовин Эаджисовна: средний балл = 3.00, троек = есть, стипендия = Социальная
NOTICE:  Студент Фарамир Итилиэнский: средний балл = 2.50, троек = есть, стипендия = Социальная
NOTICE:  === Итого: назначено 8 стипендий ===
NOTICE:  - Академических: 2
NOTICE:  - Повышенных: 3
NOTICE:  - Социальных: 3
CALL

```

Проверяем новые назначения стипендий:


```

DB_session=# SELECT
  с.фамилия || ' ' || с.имя AS студент,
  st.номер_группы,
  sch.тип_стипендии,
  sch.размер_стипендии,
  app.дата_назначения
FROM session_schema.Назначение_стипендии app
JOIN session_schema.Человек с ON app.id_человека = с.id_человека
JOIN session_schema.Студент st ON с.id_человека = st.id_человека
JOIN session_schema.Стипендия sch ON app.id_стипендии = sch.id_стипендии
WHERE st.номер_группы = 'K3241'

```

студент	номер_группы	тип_стипендии	размер_стипендии	дата_назначения
Арагорн Элессар	K3241	Повышенная	7000.00	2026-06-09
Бэггинс Фродо	K3241	Повышенная	7000.00	2026-06-09
Зеленый Леголас	K3241	Академическая	3500.00	2026-06-09
Теоден Эаджисович	K3241	Академическая	3500.00	2026-06-09
Торин Дубоцит	K3241	Социальная	2500.00	2026-06-09
Фарамир Итилиэнский	K3241	Социальная	2500.00	2026-06-09
Эовин Эаджисовна	K3241	Социальная	2500.00	2026-06-09
Эомер Эаджисович	K3241	Повышенная	7000.00	2026-06-09

(8 строк)

3. Отчисление студентов за академическую неуспеваемость.

Создаем процедуру:

```

CREATE OR REPLACE PROCEDURE session_schema.process_expulsions(
  IN p_nomer_gruppy VARCHAR(18),
  IN p_max_fails INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_student_rec RECORD;
  v_fail_count INTEGER;
  v_expelled_count INTEGER := 0;
BEGIN
  -- Проверяем существование группы
  IF NOT EXISTS (SELECT 1 FROM session_schema.Группа WHERE номер_группы =
p_nomer_gruppy) THEN
    RAISE EXCEPTION 'Группа % не найдена', p_nomer_gruppy;
  END IF;

  -- Перебираем студентов указанной группы, которые еще учатся
  FOR v_student_rec IN
    SELECT st.id_человека, с.фамилия || ' ' || с.имя AS fio
    FROM session_schema.Студент st
    JOIN session_schema.Человек с ON st.id_человека = с.id_человека
    WHERE st.номер_группы = p_nomer_gruppy
    AND st.статус = 'обучается'
  LOOP
    -- Считаем количество двоек или неспдч
    SELECT COUNT(*) INTO v_fail_count
    FROM session_schema.Сессия s
    WHERE s.id_человека = v_student_rec.id_человека
    AND (s.оценка = 2 OR s.статус_сдачи = 'не сдано');

    -- Если хвостов больше или равно двум, то отчисляем
    IF v_fail_count >= p_max_fails THEN
      RAISE NOTICE 'Студент % имеет % неуспешных попыток', v_student_rec.fio,

```

```

v_fail_count;

-- Меняем статус студента
UPDATE session_schema.Студент
SET статус = 'отчислен'
WHERE id_человека = v_student_rec.id_человека;

-- Удаляем стипендию
DELETE FROM session_schema.Назначение_стипендии
WHERE id_человека = v_student_rec.id_человека;

v_expelled_count := v_expelled_count + 1;
END IF;
END LOOP;

RAISE NOTICE ' Итого отчислено студентов: %', v_expelled_count;
END;
$$;

```

Находим студентов определенной группы, проверяем их статус и количество несдач:

```

DB_session=# SELECT
    c.фамилия,
    st.статус,
    COUNT(CASE WHEN s.оценка = 2 OR s.статус_сдачи = 'не сдано' THEN 1 END) AS количество_двоек
FROM session_schema.Студент st
JOIN session_schema.Человек c ON st.id_человека = c.id_человека
LEFT JOIN session_schema.Сессия s ON st.id_человека = s.id_человека
WHERE st.номер_группы = 'K3241'
GROUP BY c.фамилия, st.статус
ORDER BY c.фамилия;

```

фамилия	статус	количество_двоек
Арагорн	обучается	0
Бэггинс	обучается	1
Грозный	отчислен	0
Зеленый	обучается	0
Теоден	обучается	0
Торин	обучается	0
Фарамир	обучается	1
Эовин	обучается	0
Эомер	обучается	0

(9 строк)

Добавляем определенному студенту несдачи и вызываем процедуру:

```

DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
VALUES
('FAIL_1', 'STU_01', 'DUP_01', 'SCH_01', '2025/2026', 5, 2, 'не сдано', 2, '2026-01-20', '2026-01-2
0'),
('FAIL_2', 'STU_01', 'DUP_02', 'SCH_02', '2025/2026', 5, 2, 'не сдано', 2, '2026-01-21', '2026-01-2
1');
INSERT 0 2
DB_session=# CALL session_schema.process_expulsions('K3241'::VARCHAR, 2);
NOTICE:  Студент Бэггинс Фродо имеет 3 неуспешных попыток. Отчисляется...
NOTICE:  Итого отчислено студентов: 1
CALL

```

Проверяем новый список студентов, их статус и количество несдач:

```
DB_session=# SELECT
    с.фамилия,
    st.статус
FROM session_schema.Студент st
JOIN session_schema.Человек с ON st.id_человека = с.id_человека
WHERE st.номер_группы = 'K3241'
ORDER BY с.фамилия;
```

фамилия	статус
Арагорн	обучается
Бэггинс	отчислен
Грозный	отчислен
Зеленый	обучается
Теоден	обучается
Торин	обучается
Фарамир	обучается
Эовин	обучается
Эомер	обучается

(9 строк)

Триггеры:

1. Логирование изменений в таблице «Студент».

Создаем таблицу для хранения истории изменений:

```
DB_session=# CREATE TABLE session_schema.Студент_лог (
    id_лога SERIAL PRIMARY KEY,
    id_человека VARCHAR(18),
    старая_группа VARCHAR(18),
    новая_группа VARCHAR(18),
    операция VARCHAR(10),
    время_изменения TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE _
```

Создаем триггерную функцию:

```
DB_session=# CREATE OR REPLACE FUNCTION session_schema.fn_лог_студент()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO session_schema.Студент_лог (id_человека, новая_группа, операция)
        VALUES (NEW.id_человека, NEW.номер_группы, 'INSERT');
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO session_schema.Студент_лог (id_человека, старая_группа, новая_группа, операция)
        VALUES (NEW.id_человека, OLD.номер_группы, NEW.номер_группы, 'UPDATE');
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO session_schema.Студент_лог (id_человека, старая_группа, операция)
        VALUES (OLD.id_человека, OLD.номер_группы, 'DELETE');
        RETURN OLD;
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Создаем триггер:

```
DB_session=# CREATE TRIGGER tr_лог_студент
AFTER INSERT OR UPDATE OR DELETE ON session_schema.Студент
FOR EACH ROW
EXECUTE FUNCTION session_schema.fn_лог_студент();
CREATE TRIGGER
```

Переводим студента в другую группу:

```
DB_session=# UPDATE session_schema.Студент
SET номер_группы = 'K3239'
WHERE id_человека = 'STU_02';
UPDATE 1
```

Смотрим, что попало в лог:

```
DB_session=# SELECT * FROM session_schema.Студент_лог ORDER BY id_лога DESC;
```

id_лога	id_человека	старая_группа	новая_группа	операция	время_изменения
1	STU_02	K3241	K3239	UPDATE	2026-06-09 18:25:28.462956

(1 строка)

2. Проверка статуса сдачи и оценки.

Создаем триггерную функцию:

```
DB_session=# CREATE OR REPLACE FUNCTION session_schema.fn_validate_session()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.статус_сдачи = 'сдано' THEN
        IF NEW.оценка IS NULL OR NEW.оценка < 3 THEN
            RAISE EXCEPTION 'Ошибка бизнес-логики: При статусе "сдано" оценка должна быть не меньше
3';
        END IF;
    END IF;
    IF NEW.статус_сдачи = 'не сдано' THEN
        IF NEW.оценка IS NOT NULL AND NEW.оценка > 2 THEN
            RAISE EXCEPTION 'Ошибка бизнес-логики: При статусе "не сдано" оценка не может быть боль
ше 2';
        END IF;
    END IF;

    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Создаем триггер:

```
DB_session=# CREATE TRIGGER tr_validate_session
BEFORE INSERT OR UPDATE ON session_schema.Сессия
FOR EACH ROW
[EXECUTE FUNCTION session_schema.fn_validate_session();
CREATE TRIGGER
```

Пытаемся вставить статус "не сдано" с оценкой 5:

```
DB_session=#

INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расписания, учебн
ый_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
[VALUES ('TEST_FAIL_1', 'STU_01', 'DUP_01', 'SCH_01', '2025/2026', 5, 5, 'не сдано', 1, CURRENT_DATE
, CURRENT_DATE);
ERROR:  Ошибка бизнес-логики: При статусе "не сдано" оценка не может быть больше 2
KONTEXT:  PL/pgSQL function session_schema.fn_validate_session() line 10 at RAISE
```

Вставляем корректные данные:

```
DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
VALUES ('TEST_OK_1', 'STU_01', 'DUP_01', 'SCH_01', '2025/2026', 5, 4, 'сдано', 1, CURRENT_DATE, CUR
RENT_DATE);
INSERT 0 1
```

3. Автоматическая очистка данных при отчислении.

Создаем триггерную функцию и сам триггер:

```
DB_session=# CREATE OR REPLACE FUNCTION session_schema.fn_cleanup_expelled_student()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.статус != 'отчислен' AND NEW.статус = 'отчислен' THEN

        -- Удаляем все записи о сессиях этого студента
        DELETE FROM session_schema.Сессия
        WHERE id_человека = OLD.id_человека;

        -- Удаляем назначение стипендии
        DELETE FROM session_schema.Назначение_стипендии
        WHERE id_человека = OLD.id_человека;

        RAISE NOTICE 'Студент % отчислен. Все записи о сессиях и стипендиях автоматически удалены.'
, OLD.id_человека;
    END IF;

    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
DB_session=# CREATE TRIGGER tr_cleanup_expelled
AFTER UPDATE ON session_schema.Студент
FOR EACH ROW
[EXECUTE FUNCTION session_schema.fn_cleanup_expelled_student();
CREATE TRIGGER
```

Проверяем сессии определенного студента:

```
DB_session=# SELECT id_сессии, id_человека, оценка, статус_сдачи
FROM session_schema.Сессия
[WHERE id_человека = 'STU_02';
id_сессии | id_человека | оценка | статус_сдачи
-----+-----+-----+-----
SES_02    | STU_02        |      4 | сдано
(1 строка)
```

Отчисляем студента и проверяем:

```
DB_session=# UPDATE session_schema.Студент
SET статус = 'отчислен'
[WHERE id_человека = 'STU_02';
NOTICE: Студент STU_02 отчислен. Все записи о сессиях и стипендиях автоматически удалены.
UPDATE 1
DB_session=# SELECT id_сессии, id_человека, оценка, статус_сдачи
FROM session_schema.Сессия
[WHERE id_человека = 'STU_02';
id_сессии | id_человека | оценка | статус_сдачи
-----+-----+-----+-----
(0 строк)
```

4. Проверка последовательности и лимита попыток.

Создаем триггерную функцию и сам триггер:

```
DB_session=# CREATE OR REPLACE FUNCTION session_schema.fn_check_attempt_sequence()
RETURNS TRIGGER AS $$
DECLARE
    v_max_attempt INTEGER;
BEGIN
    -- Находим максимальный номер попытки для этого студента и дисциплины
    SELECT COALESCE(MAX(номер_попытки), 0) INTO v_max_attempt
    FROM session_schema.Сессия
    WHERE id_человека = NEW.id_человека
    AND id_дисциплины_в_УП = NEW.id_дисциплины_в_УП;

    -- Номер попытки должен быть строго больше предыдущего
    IF NEW.номер_попытки <= v_max_attempt THEN
        RAISE EXCEPTION 'Нарушение логики: Попытка №% не может быть меньше или равна уже существующей (макс. %)', NEW.номер_попытки, v_max_attempt;
    END IF;

    -- Лимит в 3 попытки
    IF NEW.номер_попытки > 3 THEN
        RAISE EXCEPTION 'Превышен лимит попыток сдачи (максимум 3)';
    END IF;

    RETURN NEW;
END;
[ $$ LANGUAGE plpgsql;
CREATE FUNCTION
DB_session=# CREATE TRIGGER tr_check_attempt_sequence
BEFORE INSERT ON session_schema.Сессия
FOR EACH ROW
[EXECUTE FUNCTION session_schema.fn_check_attempt_sequence();
CREATE TRIGGER
```

Пытаемся вставить дублирующую попытку:

```
DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
[VALUES ('TEST_DUP_1', 'STU_01', 'DUP_01', 'SCH_01', '2025/2026', 5, 3, 'сдано', 1, CURRENT_DATE, CU
RRENT_DATE);
ERROR:  Нарушение логики: Попытка №1 не может быть меньше или равна уже существующей (макс. 2)
КОНТЕКСТ:  PL/pgSQL function session_schema.fn_check_attempt_sequence() line 13 at RAISE
```

Вставляем корректные данные:

```
DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
VALUES ('TEST_OK_2', 'STU_01', 'DUP_02', 'SCH_02', '2025/2026', 5, 4, 'сдано', 2, CURRENT_DATE, CU
RRENT_DATE);
INSERT 0 1
```

5. Проверка согласованности даты сессии и расписания.

Создаем триггерную функцию и сам триггер:


```

DB_session=# CREATE OR REPLACE FUNCTION session_schema.fn_check_session_date()
RETURNS TRIGGER AS $$
DECLARE
    v_schedule_date DATE;
BEGIN
    -- Получаем дату проведения из таблицы Расписание
    SELECT дата_проведения INTO v_schedule_date
    FROM session_schema.Расписание
    WHERE id_расписания = NEW.id_расписания;

    -- Проверяем совпадение даты начала сессии с датой в расписании
    IF NEW.дата_начала != v_schedule_date THEN
        RAISE EXCEPTION 'Ошибка согласованности: Дата начала сессии (%) не совпадает с датой провед
ения по расписанию (%)',
            NEW.дата_начала, v_schedule_date;
    END IF;

    RETURN NEW;
END;
[$$ LANGUAGE plpgsql;
CREATE FUNCTION
DB_session=# CREATE TRIGGER tr_check_session_date
BEFORE INSERT OR UPDATE ON session_schema.Сессия
FOR EACH ROW
[EXECUTE FUNCTION session_schema.fn_check_session_date();
CREATE TRIGGER

```

Пытаемся вставить сессию с неправильной датой:

```

DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
[VALUES ('TEST_DATE_1', 'STU_02', 'DUP_01', 'SCH_01', '2025/2026', 5, 4, 'сдано', 1, '2026-01-20', '2
026-01-20');
ERROR:  Ошибка согласованности: Дата начала сессии (2026-01-20) не совпадает с датой проведения по
расписанию (2026-01-15)
КОНТЕКСТ:  PL/pgSQL function session_schema.fn_check_session_date() line 12 at RAISE

```

Вставляем корректные данные:

```

DB_session=# INSERT INTO session_schema.Сессия (id_сессии, id_человека, id_дисциплины_в_УП, id_расп
исания, учебный_год, семестр, оценка, статус_сдачи, номер_попытки, дата_начала, дата_конца)
[VALUES ('TEST_DATE_OK', 'STU_02', 'DUP_01', 'SCH_01', '2025/2026', 5, 4, 'сдано', 1, '2026-01-15', '
2026-01-15');
INSERT 0 1

```

Вывод:

В ходе выполнения лабораторной работы были приобретены практические навыки создания и использования процедур, функций и триггеров в СУБД PostgreSQL. Разработаны три процедуры для БД «Сессия»: зачисление студента на пересдачу с контролем количества попыток, автоматическое начисление стипендий по результатам сессии и отчисление студентов за академическую неуспеваемость. Созданы пять оригинальных триггеров, обеспечивающих логирование изменений в таблице «Студент», проверку бизнес-логики соответствия оценок и статусов сдачи, автоматическую очистку будущих сессий при отчислении, контроль последовательности и лимита попыток сдачи, а также проверку согласованности дат сессии и расписания.